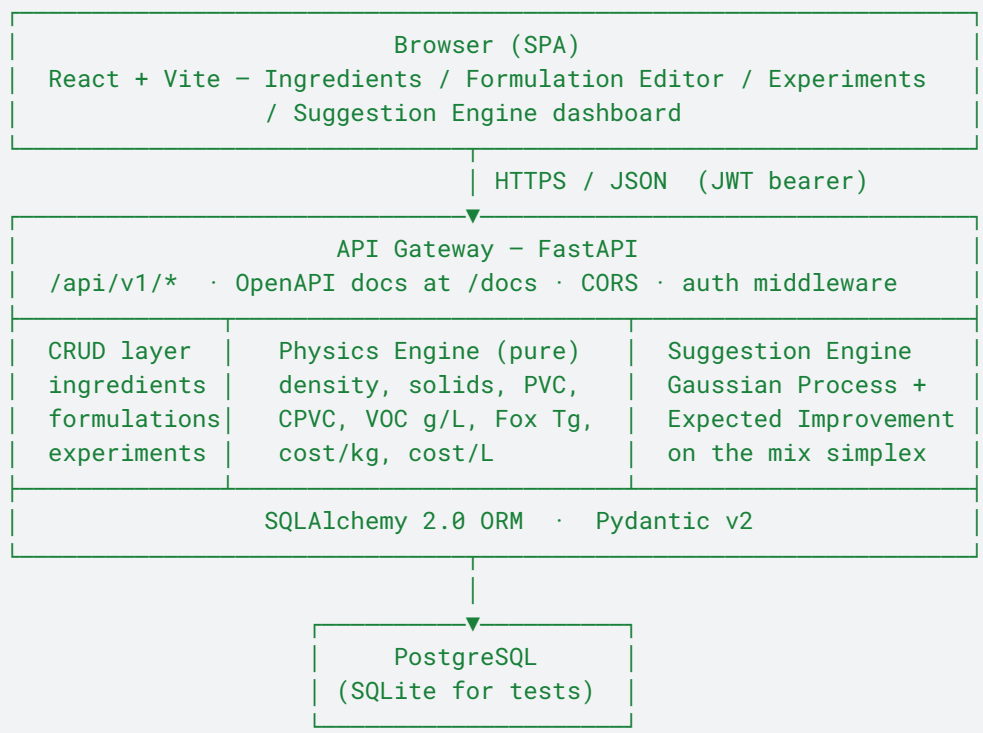
Amphora — Materials Formulation Platform

Mission: an R&D operating system for coatings. Scientists define ingredients, compose formulations, log experiments, and the platform computes physics-derived properties and proposes the next formulations to try.

---

1. System Architecture

None



Design principles:

- **Physics and ML are pure, stateless modules** (`services/physics.py`, `services/optimizer.py`). No DB imports. Unit-testable, swappable, and extractable into workers when compute grows.
- **Thin routers, fat services.** Routers do I/O validation and auth; domain logic lives in services.

- **Formulations are immutable once experiments reference them.** Iteration happens by cloning to a new version (lineage via `parent_id`), preserving full provenance — non-negotiable for R&D reproducibility and future patent filings.
- **API versioned at `/api/v1`** so the mobile/lab-instrument clients that come later don't break.

2. File Structure

```
None
amphora/
├── docker-compose.yml            # postgres + api + web, one command up
├── README.md
├── docs/ARCHITECTURE.md
├── backend/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── app/
│   │   ├── main.py               # app factory, router mounting, startup
│   │   ├── core/
│   │   │   ├── config.py         # pydantic-settings, 12-factor env config
│   │   │   ├── db.py             # engine, session dependency
│   │   │   └── security.py       # bcrypt + JWT
│   │   ├── models.py             # SQLAlchemy ORM (single module at MVP scale)
│   │   ├── schemas.py            # Pydantic request/response models
│   │   ├── api/
│   │   │   ├── deps.py           # get_db, get_current_user
│   │   │   ├── auth.py           # token, me
│   │   │   ├── ingredients.py    # CRUD
│   │   │   ├── formulations.py   # CRUD + clone + computed properties
│   │   │   ├── experiments.py    # batches + measurements
│   │   │   └── suggestions.py    # optimizer endpoint
│   │   ├── services/
│   │   │   ├── physics.py        # coating physics (pure functions)
│   │   │   └── optimizer.py      # GP + EI candidate generation (pure)
│   │   └── seed.py               # demo ingredients, formulation, results
│   └── tests/
│       ├── test_physics.py
│       ├── test_optimizer.py
│       └── test_api.py
└── frontend/
    ├── Dockerfile                # build + nginx serve, /api proxy
    ├── package.json / vite.config.js / index.html
    └── src/
        ├── main.jsx / App.jsx    # router, auth guard, layout
        ├── api.js                # fetch client, token handling
        ├── styles.css
        └── pages/
```

```
        ├── Login.jsx
        ├── Ingredients.jsx
        ├── Formulations.jsx  # list + lineage
        ├── FormulationEditor.jsx   # live computed properties
        ├── Experiments.jsx   # batches + measurements entry
        └── Suggest.jsx       # optimizer UI
```

3. Database Schema

```
SQL
users            (id PK, email UNIQUE, hashed_password, full_name, role,
created_at)

ingredients      (id PK, name UNIQUE, cas_number, supplier, role,        --
binder|pigment|filler|solvent|cosolvent|additive
                  density_g_ml, solids_fraction,                          -- 0..1
(nonvolatile)
                  tg_c,                                                   --
polymer Tg, nullable
                  oil_absorption,                                         -- g
oil/100 g pigment, nullable
                  is_water BOOL, voc_exempt BOOL,
                  cost_per_kg, notes, created_at)

formulations     (id PK, code UNIQUE, name, description,
                  status,                                                 --
draft|active|archived
                  version INT, parent_id FK→formulations,                 --
lineage tree
                  created_by FK→users, created_at)

formulation_components
                 (id PK, formulation_id FK CASCADE, ingredient_id FK,
                  mass_fraction NUMERIC,                                   -- Σ = 1
enforced in service layer
                  UNIQUE(formulation_id, ingredient_id))

experiments      (id PK, formulation_id FK, batch_code UNIQUE, status,   --
planned|prepared|tested|failed
                  notes, created_by FK→users, prepared_at, created_at)

measurements     (id PK, experiment_id FK CASCADE,
                  property_name,                                          -- e.g.
gloss_60, adhesion_mpa, scrub_cycles
                  value NUMERIC, unit, method, measured_at)
```

Indexes: FKs, `measurements.property_name`, `formulations.parent_id`. Computed properties are **never stored** — they derive from composition and are recomputed on read (cacheable later; correctness first).

4. API Surface (`/api/v1`)

Method	Path	Purpose
POST	`/auth/token`	OAuth2 password flow → JWT
GET	`/auth/me`	Current user
GET/POST	`/ingredients` · `/ingredients/{id}` (GET/PUT/DELETE)	Ingredient library
GET/POST	`/formulations` · `/formulations/{id}` (GET/PUT/DELETE)	Recipes (components inline)
POST	`/formulations/{id}/clone`	New version, lineage preserved
GET	`/formulations/{id}/properties`	**Physics engine output**
POST	`/formulations/preview-properties`	Physics for an unsaved composition (live editor)
GET/POST	`/experiments` · `/experiments/{id}`	Batches
POST	`/experiments/{id}/measurements`	Log test results
POST	`/suggestions`	Propose next formulations for an objective
GET	`/health`	Liveness

5. Physics Engine (v1 models)

All from composition + ingredient constants; SI-adjacent coating conventions.

- **Wet density** $\rho = 1 / \Sigma(w_i/\rho_i)$
- **Solids** by weight $\Sigma\, w_i \cdot s_i$; by volume from per-ingredient solid volumes
- **PVC** = V_pigment+filler / (V_pigment+filler + V_binder_solids)
- **CPVC** $\approx \Sigma v_i \cdot (1/\rho_i)/(1/\rho_i + OA_i/93.5)$ over pigments/fillers (oil-absorption model, linseed ρ=0.935)
- **VOC (g/L)** = m_volatile_organics / (V_paint − V_water − V_exempt) — US EPA Method 24 convention
- **Binder Tg** — Fox equation over binder solids: $1/Tg = \Sigma(w_i/Tg_i)$, Kelvin
- **Cost** /kg and /L
- Warnings: PVC > CPVC (above critical → porosity/loss of gloss), high VOC, composition ≠ 100%

6. Suggestion Engine

Objective: given past `(composition → measured property)` pairs, propose the next batch.

- < 5 observations → **space-filling**: Dirichlet sampling on the constrained simplex (bounds per ingredient respected, renormalized).
- ≥ 5 → **Gaussian Process** (RBF kernel, pure NumPy) fit on compositions; **Expected Improvement** acquisition ranks ~2,000 candidate mixes; top-N returned with predicted mean ± σ and physics properties.
- Constraints: per-ingredient min/max mass fraction; Σ = 1 by construction.

Deliberately dependency-light (NumPy only). Swap-in path: BoTorch/Ax behind the same `propose()` interface when multi-objective and batch acquisition are needed.

7. Scaling Path (post-MVP)

1. **Compute**: move physics/optimizer behind a task queue (Celery/Arq + Redis) when property models get expensive (e.g., MD simulation adapters).
2. **Data**: Alembic migrations (schema is migration-ready), read replicas, TimescaleDB for instrument time-series.
3. **Models**: property-prediction registry (`services/` → model microservice), experiment-design campaigns table, multi-objective optimization.
4. **Org**: multi-tenant via `org_id` on all tables + row-level security; SSO (OIDC).
5. **Lab**: instrument ingestion service (CSV/SiLA2) posting to `/measurements`.
6. **Audit**: append-only event log table for 21 CFR Part 11-style compliance when customers demand it.